databricks

Spark at scale: Do's & Don'ts



Spark Do's & Don'ts workshop

©2026 Databricks Inc. — All rights reserved

Agenda

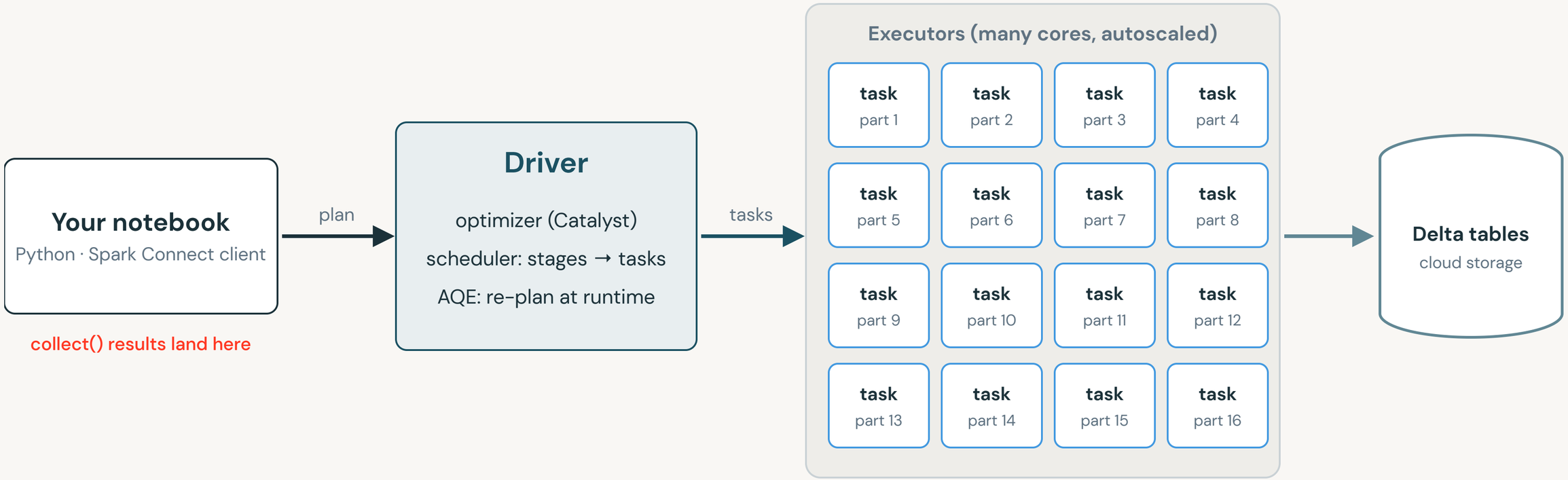
Efficient large-scale computing on Databricks

- How Spark runs your code: driver, executors, lazy plans
- Where the time goes: shuffles, joins, data layout
- What serverless and Photon do for you, and what they can't
- Five ways to leverage the engine, and the anti-patterns to avoid
- The workshop and the Genie Code challenge

©2026 Databricks Inc. — All rights reserved

Anatomy of a Spark application

On serverless, your notebook is a client and Databricks runs the engine



Only what you explicitly ask for comes back to the notebook. `collect()` and `toPandas()` ask for everything.

©2026 Databricks Inc. — All rights reserved

Scale is a parallelism problem

The same question, answered two ways

Summing 100M order lines

collect() to one Python process



≈ 13 min

aggregate in Spark, in parallel



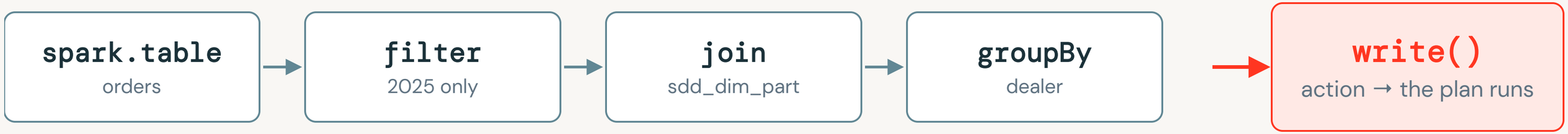
Measured in the workshop on serverless (the collect() time is projected from a 4% slice).

- Spark splits data into **partitions** and works on all of them at once, on many cores
- Efficient code keeps every core busy and **moves as little data as possible**

©2026 Databricks Inc. — All rights reserved

Lazy evaluation

Transformations describe the work, actions run it



transformations only build a plan: nothing has run yet

Transformations (lazy)

```
select · filter · withColumn · join · groupBy · orderBy · union
```

Actions (run the whole plan, every time)

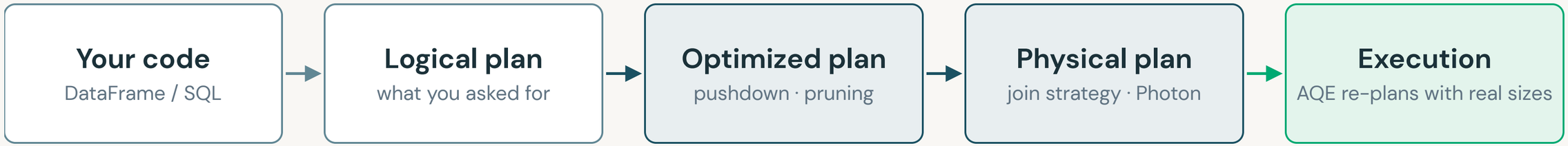
```
count · collect · display · toPandas · write · isEmpty
```

Five debug `count()` calls in a pipeline = the pipeline runs five times.

©2026 Databricks Inc. — All rights reserved

From DataFrame code to a physical plan

The optimizer rewrites what you wrote, if it can see it



The optimizer can rewrite

– filters pushed down to the file scan

– only the columns you use are read

– small tables broadcast instead of shuffled

...but it can't see inside

– Python UDFs and pandas apply()

– lists you collect() and paste back in

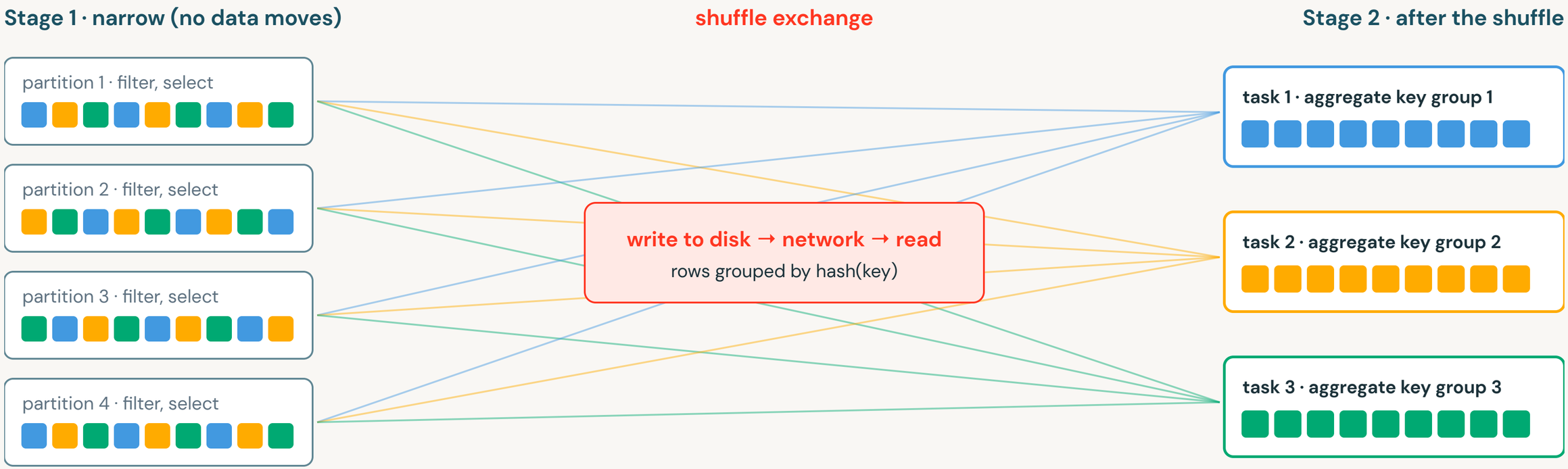
– loops that build 300-level-deep plans

Write code the optimizer can read: built-in functions, joins and filters, not Python black boxes.

©2026 Databricks Inc. — All rights reserved

Partitions, stages and the shuffle

Narrow operations stream; wide operations move data



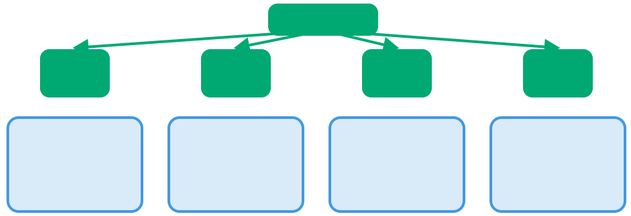
- **Narrow** (filter, select, withColumn): each partition on its own, no data moves
- **Wide** (groupBy, join, orderBy, window): rows with the same key must meet, so they cross the network
- One partition = one task = one core: a window with no `partitionBy` runs on a single core

©2026 Databricks Inc. — All rights reserved

How Spark joins

The strategy, not the syntax, decides the cost

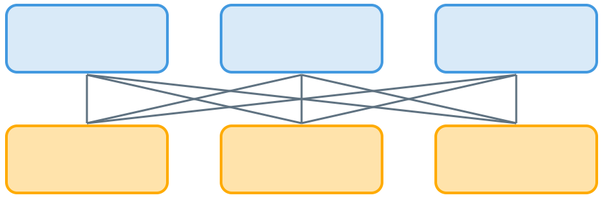
Broadcast hash join



small side copied to every executor

the big side never moves

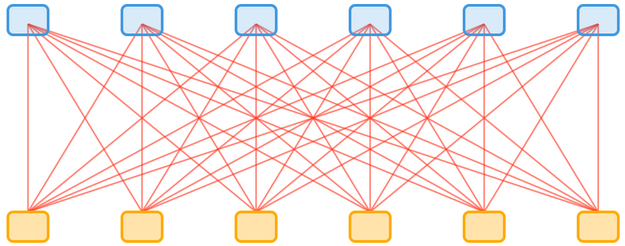
Shuffle join



both sides re-partitioned by key

fine for big ⋈ big, costs a shuffle

Nested loop / cartesian



no usable equality condition

every row × every row

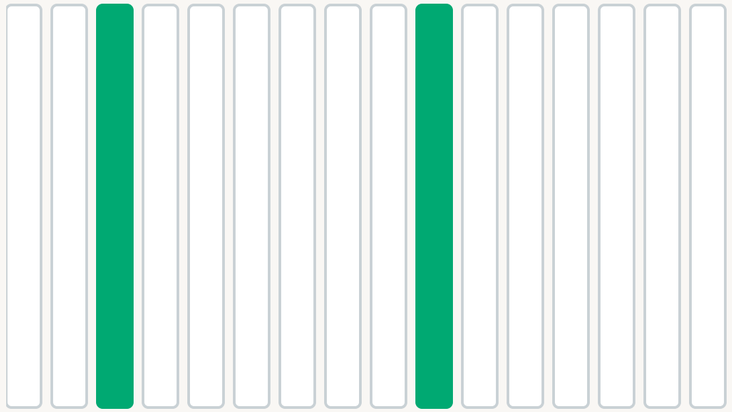
Joins on a non-unique key multiply rows. In the workshop, joining orders to an SCD2 history doubled revenue, with no error.

©2026 Databricks Inc. — All rights reserved

Data layout is performance

The fastest data is the data you never read

Columns: read only what you use



2 of 300 columns read from Parquet/Delta

toPandas() first = all 300 read and shipped

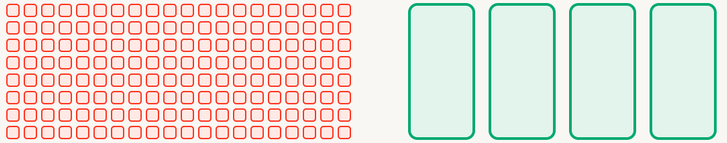
Files: skip what can't match

- file 1 min/max: Jan–Feb
- file 2 min/max: Mar–Apr
- file 3 min/max: May–Jun
- file 4 min/max: Jul–Aug
- file 5 min/max: Sep–Oct
- **file 6 min/max: Nov–Dec**

`WHERE order_date >= '2025-11-01'`

liquid clustering keeps ranges tight

Size: fewer, right-sized files



same rows: 6.7 s vs 1.1 s

OPTIMIZE / predictive optimization

aim for ~100 MB – 1 GB per file

Select the columns you need, filter on clustered columns, keep files right-sized: read less, finish sooner.

©2026 Databricks Inc. — All rights reserved

What serverless does for you

…and what is still up to your code

The platform handles

- Adaptive query execution: broadcast decisions, partition sizes, skew
- Photon: vectorized, native execution of built-in operators
- Compute sizing and autoscaling
- Optimized writes, auto compaction, predictive optimization

Your code still decides

- **Where** the work runs: executors or one driver process
- **Whether** the optimizer can see your intent
- **How often** the same work is repeated
- **How** data is laid out and keyed

```
spark.conf.set("spark.executor.memory", "16g")   # → [CONFIG_NOT_AVAILABLE] on serverless
```

©2026 Databricks Inc. — All rights reserved

THE DO'S AND DON'TS

Five ways to leverage the engine

1. Keep the work on the executors
2. Let the optimizer see your intent
3. Don't do the work twice
4. Respect the data layout
5. Know your keys and your platform

©2026 Databricks Inc. — All rights reserved

1 · Keep the work on the executors

The driver is one machine. Anything that funnels rows through it loses the parallelism you are paying for.

DON'T

- `collect()` / `toPandas()` on big DataFrames
- a Python loop running one query per key
- one `INSERT` per row
- `display()` left in production jobs

DO

- aggregate, filter and `limit` in Spark, then bring back the small result
- one `groupBy` or join over all keys
- one batch append, or `MERGE INTO`
- one action (the write); row counts from `DESCRIBE HISTORY`

collect() 100M rows (≈ 49 GB on a 15 GB driver)

≈ 13 min → 1.6 s

query per dealer ×500

≈ 7 min → 0.8 s

INSERT per row ×300

≈ 6 min → 1.2 s

©2026 Databricks Inc. — All rights reserved

2 · Let the optimizer see your intent

Built-in functions and declarative joins can be optimized and run in Photon. Opaque Python and giant literal lists can't.

DON'T

- Python UDFs for arithmetic and if/else
- `withColumn` in a loop over hundreds of columns
- `isin(<100k collected IDs>)`
- `BETWEEN` joins with no usable equality

DO

- built-in functions (› pandas UDF › Python UDF)
- `withColumns({...})` or one `select`
- a left semi join
- the `range_join` hint, or a bucket key

Python UDF

≈ 120 s → 1.4 s

withColumn() loop

≈ 83 s → 1.0 s

range join

33 s → 1.4 s

©2026 Databricks Inc. — All rights reserved

3 · Don't do the work twice

Every action re-runs the plan from the source. Every full reload re-reads all of history.

DON'T

- `count()` / `display()` after every step
- `if df.count() > 0`
- re-reading the whole landing zone each run

DO

- end the pipeline in one write; `cache()` only when you truly reuse
- `if not df.isEmpty()`
- Auto Loader or declarative pipelines (incremental)

debug actions per step

31 s → 14 s

count() > 0

11.6 s → 0.7 s

full reload vs Auto Loader

14 s and growing → 8 s

©2026 Databricks Inc. — All rights reserved

4 · Respect the data layout

File count, file size and partitioning decide how much work every later read does.

DON'T

- thousands of tiny files
- `inferSchema` on CSV and JSON
- `partitionBy` on high-cardinality columns
- `coalesce(1)` before a big write

DO

- right-sized files; `OPTIMIZE` / predictive optimization
- an explicit schema, or Auto Loader schema hints
- liquid clustering (`CLUSTER BY`)
- parallel writes; aggregate first if you need one file

partitionBy(date, dealer)
≈ 23 min → 13 s

coalesce(1) write
≈ 89 s → 12.5 s

inferSchema
17.6 s → 4.3 s

©2026 Databricks Inc. — All rights reserved

5 · Know your keys and your platform

Keys decide parallelism and correctness. Cluster-era tuning doesn't apply on serverless.

DON'T

- `Window.orderBy(...)` with no `partitionBy`
- joining on a key that isn't unique
- `spark.conf.set(...)` tuning, `sparkContext`, RDDs

DO

- `Window.partitionBy(key).orderBy(...)`
- one row per key: `is_current` or point-in-time
- DataFrame APIs, targeted hints; let AQE tune

window without partitionBy

≈ 41 s → 7.4 s

join on SCD2 history

+100% revenue → exact

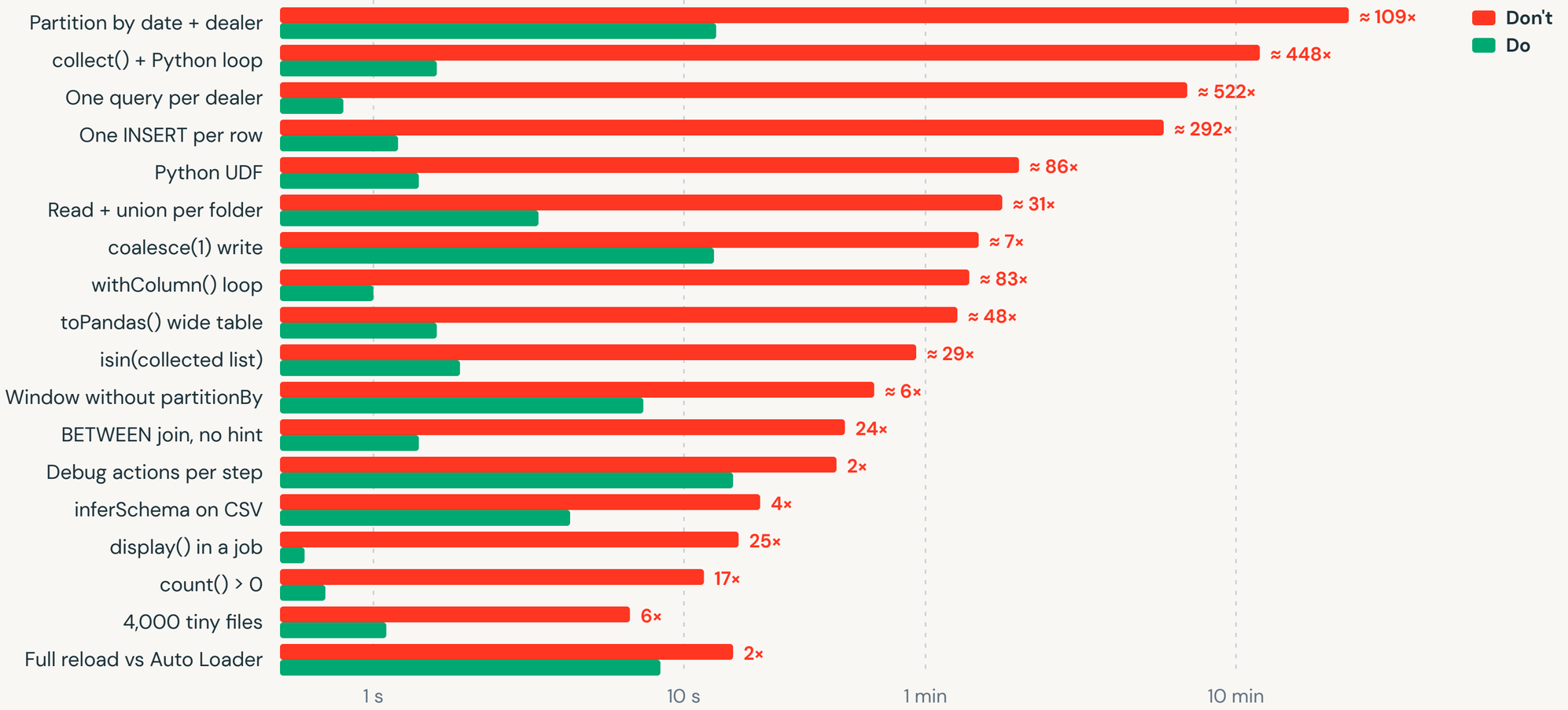
spark.executor.memory

rejected → not needed

©2026 Databricks Inc. — All rights reserved

Same answers, very different costs

Every Don't/Do pair from the workshop, measured on serverless



Don’t vs Do, log scale. ≈ marks a Don’t that ran on a slice of the workload and was projected to full size.

©2026 Databricks Inc. — All rights reserved

The workshop

Clone the repo into your workspace and set one line

```
# CONFIG.py (top of the repo): the only edit
CATALOG       = "dbx_workshop_dev"
SOURCE_SCHEMA = "shared"   # instructor-built, read-only
USER_SCHEMA   = "jane_doe" # your own schema
```

- Instructor, beforehand: `instructor_setup/` generates the shared data and creates each participant's schema
- `01_bronze_to_gold_lab` : bronze → silver → gold, SQL and a dashboard, hands-on
- `02_spark_dos_and_donts` : each topic is a **why**, then a timed **Don't** and **Do**; every section can resume after a restart

Genie Code challenge

A realistic dealer-scorecard job with **nine** anti-patterns from this deck. It gives the right answer, slowly.

- Ask Genie Code to find and fix them
- Paste the result; a check cell verifies the output
- Reference: ~2 min → **6 s**

©2026 Databricks Inc. — All rights reserved



©2026 Databricks Inc. — All rights reserved